USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Persistence, and a Read Defect That Looked Like the Wrong Thing

UM-NATOS-018

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-018	1.0 · 2026-08-15	**Complete, verified on hardware**	Internal

1. Abstract

nat-os now has state that outlives a power cycle. A SPI flash driver writes a checksummed record into a dedicated sector, and the kernel loads it before the scheduler starts.

The mechanism is small. The report is mostly about the defect that sat in front of it for the whole of the work: **every flash read returned the true value shifted right by one bit**, consistently, on every transaction. The chip ID read `0x34200B` where the part reports `0x684016`, which is exactly that value shifted once.

The cause was the SPI clock divider — inherited from the bootloader rather than set. Three properties of the defect made it point somewhere else, and this report records them because each is a general trap, not a flash-specific one:

- a *timing* fault that is perfectly consistent presents as a *framing* fault
- two hypotheses were tested against a board that had never been reflashed, so a working theory and a broken one produced identical evidence
- the fix that eventually worked had already been tried, and discarded, on the strength of one of those untested runs

This also closes the cache-off hazard flagged in UM-NATOS-011 §4 — by a different route than that report anticipated.

2. What was built

Component	Role
`kernel/flash.c`	SPI1 user-mode read, sector erase, page program
`kernel/store.c`	One checksummed record: magic, version, boots, frames
`kernel/linker.ld`	`.dram.rodata` extended to `flash.o` and `store.o`

The record is validated whole. A magic word and version identify it; a checksum over the payload rejects one torn by a reset mid-write. A first run and a corrupt sector are deliberately indistinguishable — both reset to defaults — because a partially-believed record is worse than no record.

2.1 Where it lives

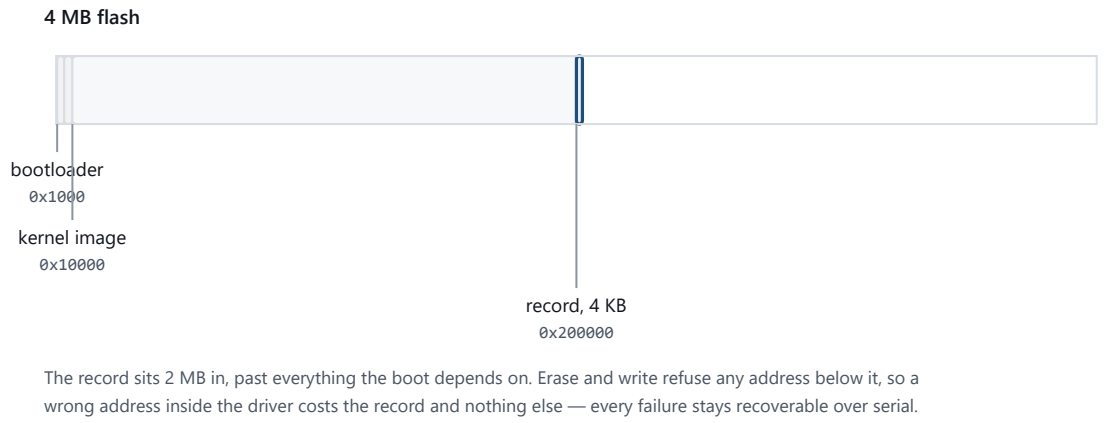

Figure 1. *The record's position in flash. Nothing the boot depends on lies above it, which is what keeps a driver defect recoverable.*

The data region starts at 2 MB, past the bootloader at `0x1000`, the partition table at `0x8000`, and the kernel image at `0x10000`. `flash_erase_sector()` and `flash_write()` both refuse any address below `FLASH_DATA_ADDR`.

This is not defence against a subtle bug. It is defence against the specific failure where a wrong constant erases the bootloader and the board stops enumerating over USB — which turns a five-minute fix into a recovery job. Every failure in this driver stays recoverable over serial.

2.2 A boot counter first, deliberately

The record's first field is a boot count, and the reason is methodological. Persistence is unusually easy to *believe* you have: the value read back is the value just written, whether or not it ever reached the chip. A RAM variable passes that test perfectly.

A counter that survives a power cycle cannot.

The second field is stronger still. `frames` accumulates raycaster frames **across** boots, so it can only be correct if the previous value was read back correctly *and added to*. A boot counter proves a write reached the chip; a running total proves the read path too. That distinction is what made the one-bit shift visible at all — the counter stuck at 1 was the symptom that started the investigation.

3. The cache hazard, closed differently than planned

UM-NATOS-011 §4 identified this as the thing that would break first:

> Any future code that writes flash must disable the cache while doing so. During that window, *any* access to `.rodata` faults — including a string in an interrupt handler, and including the panic handler's own messages.

It broke exactly as predicted, on the first version of the driver, and the presentation was memorable: every string literal in the kernel began printing as `0xFF` immediately after the first flash operation. The console filled with `??????????`.

The cause was not a cache-off window. It was worse and simpler — **the driver reconfigured SPI1 and walked away**. SPI1 shares the flash bus with the cache's SPI0, and those registers are how the cache issues its own

reads. Leaving them altered left the cache unable to read flash at all.

The fix is to save and restore all five controller registers — `USER`, `USER1`, `USER2`, `CTRL`, `CTRL2`, later `CLOCK` — unconditionally, **including on the failure path**. A driver that restores state only when it succeeds has simply moved the hazard to the error case, where it is harder to find.

3.1 Why the cache is never disabled

The planned mitigation was to disable the cache around flash writes, as ESP-IDF does. nat-os does not. It makes the dangerous reads impossible instead:

- all kernel code executes from IRAM, so instruction fetch never touches flash
- `flash.o`, `store.o`, `panic.o`, `uart.o` and `watchdog.o` have their `.rodata` placed in DRAM by the linker, so nothing on this path reads a flash-mapped address
- interrupts are masked for the whole operation, so no handler can run and touch one either

A cache *hit* is harmless — it never reaches the chip. Only a miss would, and with no flash-mapped address referenced there is nothing to miss on.

The residual risk is worth stating plainly: this argument depends on every function reachable from the driver keeping its read-only data out of flash. The linker rule selects **by object file** rather than by annotation precisely so that adding a string to one of these files cannot quietly break it. An annotation can be forgotten on a new string; a file-scoped rule cannot.

4. The read defect

4.1 Symptom

```
flash id     : 0x0034200b        expected 0x00684016
store        : initialised, boot #1, frames 0, saved
store        : initialised, boot #1, frames 0, saved     ← after a reset
```

`0x684016 >> 1 == 0x34200B`. Every read, every time, shifted right one bit with a zero entering at the top.

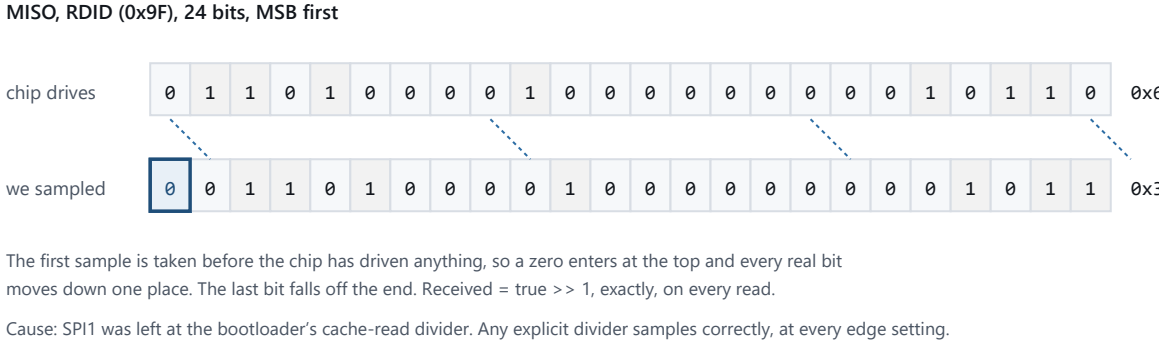

Figure 2. *The flash read defect. A sample taken one clock early inserts a leading zero, so every byte arrives as the true value shifted right once — consistently enough to look like a framing error rather than a timing one.*

4.2 Why it pointed at the command phase

A leading zero followed by the real data means the first sample was taken before the chip drove anything. Two readings of that are available:

1. **framing** — the command phase is one clock short, so data sampling begins a clock early
2. **timing** — sampling is misaligned against the data by a clock

Reading 1 was pursued first, and the reason is worth naming: the shift was *perfectly consistent*. Timing faults are expected to be marginal — to vary with temperature, or produce occasional rather than uniform corruption. A defect that reproduces bit-exactly on every transaction reads as structural.

That instinct is wrong here. A sampling point misplaced by a *whole clock* is not marginal at all; it is as deterministic as a framing error and looks identical from the outside. **Consistency does not distinguish framing from timing.** It only rules out marginality.

The specific suspect was `SPI_USER2`'s command-length field, which the driver set to `(8 - 1) << 28` on the assumption that it counts bits-minus-one. That assumption was never verified against documentation.

4.3 What ruled it out

Rather than settle the encoding question, the command and address were moved out of `USR_COMMAND` / `USR_ADDR` entirely and sent as ordinary MOSI bytes — a length field the display driver had already proven correct at every length it uses. A SPI NOR part cannot tell the difference; the phase split is a convenience of this controller, not something on the wire.

The result was **byte-for-byte identical**: `0x0034200b`.

Two structurally different transactions producing the same shift is strong evidence, and it is the single most useful measurement in this report. It did not identify the cause, but it eliminated an entire class — the extra bit arrives regardless of how the command is framed, so no command-phase register can be responsible.

4.4 The measurement that settled it

With framing eliminated, the remaining candidates were the sampling edge and the clock. Both are register fields with several plausible values, and testing them one at a time costs a build-and-flash cycle per hypothesis.

Instead, all of them were tested in a single boot — four clock dividers against four sampling-edge combinations, with the known-good answer printed alongside so the right row identifies itself:

```
flash probe  : want 0x00684016
  clk=0x00000000 edge=0x00000000 -> 0x0034200b
  clk=0x00000000 edge=0x00000040 -> 0x0034200b
  clk=0x00000000 edge=0x00000080 -> 0x00b4200b
  clk=0x00000000 edge=0x000000c0 -> 0x00b4200b
  clk=0x001c9109 edge=0x00000000 -> 0x00684016   <== MATCH
  clk=0x001c9109 edge=0x00000040 -> 0x00684016   <== MATCH
  ...
  clk=0x00009109 edge=0x000000c0 -> 0x00684016   <== MATCH
```

`clk=0` means *inherited*. The reading is unambiguous: **every explicit divider reads correctly at every edge setting; the inherited one fails at all of them.** Sixteen measurements, one flash cycle, no interpretation

required.

The bootloader leaves SPI1 configured for fast cache reads. At that rate a user transaction samples MISO one clock early. The driver now sets ~8 MHz for its own commands and restores the bootloader's value on exit. These operations are rare and small; there is nothing to gain from running the bus near its limit.

5. Two hypotheses tested against stale firmware

This is the part of the investigation that cost the most and taught the most.

`build.ps1` builds; `build.ps1 -Flash` builds *and* flashes. For a stretch of the investigation the build step ran alone, and the results were read off a board still running older firmware. The invocation used was `flash.ps1` — a script that **does not exist**. PowerShell's error went to a stream that was being filtered out of the captured output.

Two conclusions were drawn from those runs:

Hypothesis	Verdict recorded	Actually
`SPI_CTRL` fast-read mode bits	no change	genuinely no change
`SPI_CTRL2` MISO sampling delay	**disproven**	untested — never ran

The CTRL2 result was retested properly once the flash path was fixed. It *is* disproven — the conclusion was right. But it had been right by accident, and a comment in `flash.c` had already been written attributing the shift to CTRL2 as established fact. That comment was corrected.

The cost was not the wrong conclusion. It was that during the stale window, **every hypothesis returned the same answer** — the board could not have responded differently, because it was running the same firmware throughout. A sequence of identical negative results was read as "none of these are the cause" when it actually meant "no experiment has run yet".

5.1 The general form

This is the third time in this project that an instrument reported its own reading invalid and the reading beside it was believed anyway — after the frozen marker in UM-NATOS-016 and the boot banner in UM-NATOS-017 §5.

The distinguishing signature is the same each time: **a run of results that do not vary when the input does.** Three different register configurations returning bit-identical output is not evidence about registers. It is evidence that the configuration is not reaching the device.

> **Standing rule.** A negative result is only informative if the experiment demonstrably ran. When consecutive hypotheses produce identical output, suspect the harness before the theory — and verify the change reached the target, rather than inferring it from the absence of an error message.

The concrete mitigation is narrower and duller: the flash step now always shows its `Hash of data verified.` line, and that line is checked before any capture is interpreted.

6. Verification

Persistence was verified across hard resets, with the frame count accumulating:

```
--- run ~130 s so two saves land ---
  store  : loaded, boot #14, frames 256, saved
--- reset 1 ---
  store  : loaded, boot #15, frames 512, saved
--- reset 2 ---
  store  : loaded, boot #16, frames 512, saved
  flash id : 0x00684016
```

Three things are being asserted here, and each has a distinct failure mode:

- `boots` **increments on every reset** — writes reach the chip
- `frames` **grew 256 → 512 across reset 1** — the previous value was read back correctly and added to; this cannot pass on a RAM variable or a broken read
- `frames` **stayed 512 across reset 2** — the 6-second window did not reach the 256-frame save point, so nothing was written. A counter that advanced here would indicate a write not tied to the work it claims to measure

The chip ID now matches what `esptool` independently reports for the part.

No regressions in the rest of the system across a 20-second capture: `escapes=0`, `corrupt=0`, `badbuf=0`, `guards=ok`, watchdog `f/s=18/0`.

6.1 The save interval was set by measurement

The periodic save initially fired every 4096 frames, then 512, on an assumed ~8 fps. Neither ever fired inside a test window.

The renderer actually runs at **4.4 fps** with the framebuffer on — 81 frames in 18.3 seconds, read from the existing telemetry. At 512 frames that is a save every two minutes, which is why a 75-second verification run saw nothing and was briefly mistaken for a broken save path.

The interval is now 256 frames, about one save per minute. The general point: **an interval nobody exercises is an interval nobody has tested.** A save path that fires every eight minutes would have shipped unverified, and its first real execution would have been in the field.

Endurance is not a concern at this rate — a few thousand erases over a part rated for a hundred thousand, on a sector used by nothing else.

7. Metrics

Quantity	Value
Record size	20 B (magic, version, boots, frames, checksum)
Sector used	4,096 B at `0x200000`
Flash user clock	~8 MHz (80 MHz / 1 / 10)
Bootloader clock (inherited)	fast cache-read divider — **unusable for user commands**
Max transaction	64 B; page program capped at 60 B + 4 B header
Save cadence	every 256 frames ≈ 60 s at the measured 4.4 fps
Erase cost	tens of ms, interrupts masked
Registers saved/restored per transaction	6
Boots survived in test	16
Hypotheses tested against stale firmware	2

8. What this does not establish

- **No wear levelling.** One sector, erased and rewritten in place. Adequate at one save per minute; not adequate for anything application-driven.
- **No power-cut testing.** The checksum is designed to reject a torn record, but no test has actually interrupted a write mid-erase to confirm the rejection path executes. The mechanism is reasoned, not measured.
- **No application access.** Persistence is kernel-only. No syscall exposes it, so an application cannot save state — and the confinement work in UM-NATOS-016/017 would all need extending before one could.
- **No filesystem.** One fixed-layout record, not named storage.
- **No verification against a second board.** The clock finding is from one part (`0x684016`). A different flash chip may tolerate the inherited divider, which would make this defect appear board-specific to anyone who hits it.
- **The cache argument remains an argument.** §3.1 reasons that no flash-mapped read can occur during an operation. That has not been instrumented — there is no assertion that would fire if a future edit broke it.

9. References

- UM-NATOS-011 §4 — the cache-off hazard predicted there, closed in §3 here
- UM-NATOS-011 §2.2 — `.rodata` placement, extended to `flash.o` and `store.o`
- UM-NATOS-016 §3 — the frozen marker block, first instance of the §5.1 pattern
- UM-NATOS-017 §5 — the self-erasing capture, second instance
- UM-NATOS-015 §5 — the SPI2 user-mode path this driver's framing borrows from
- `kernel/flash.c` — `spi1_xfer()`, register save/restore, `FLASH_USER_CLOCK`

- `kernel/store.c` — record layout, checksum, load/save